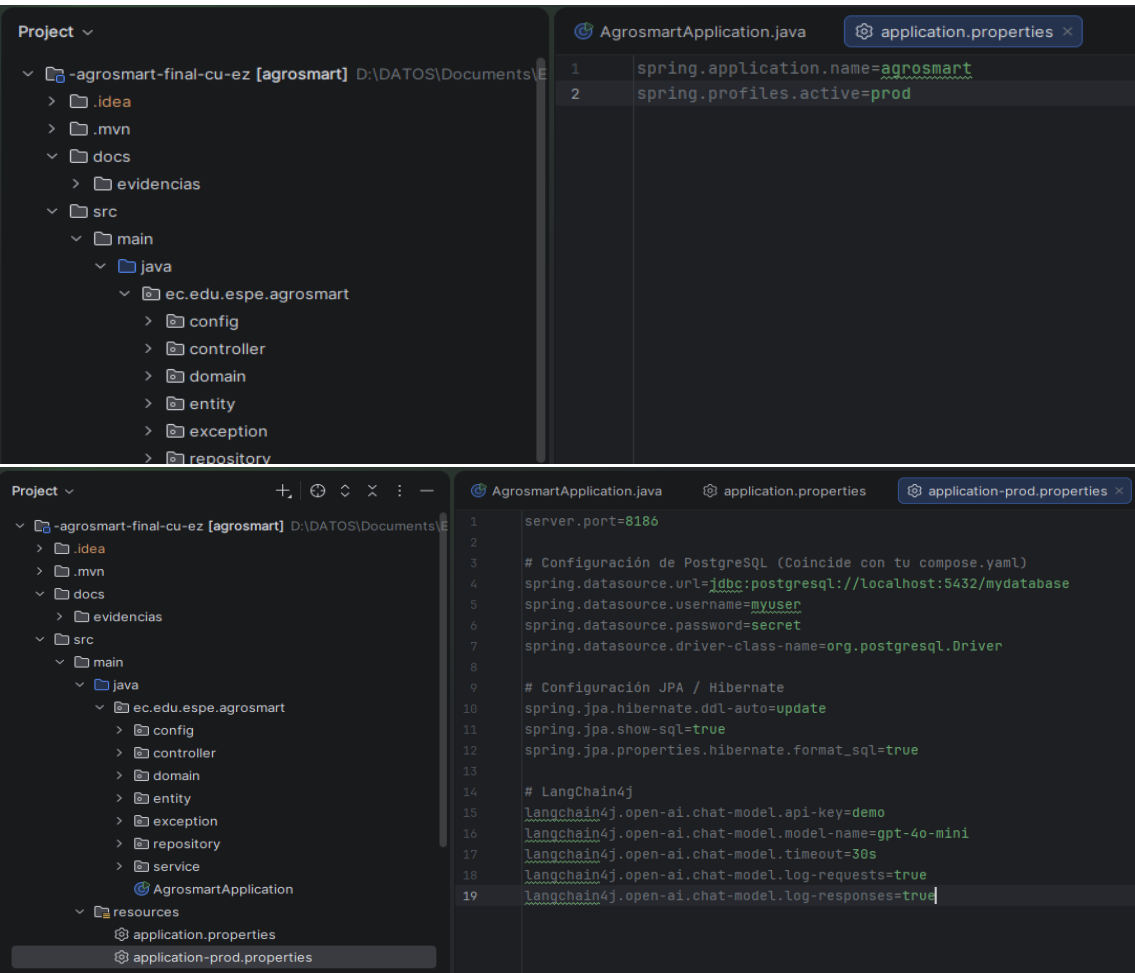


El proyecto corre en el puerto 8186 con PostgreSQL y Docker Compose.



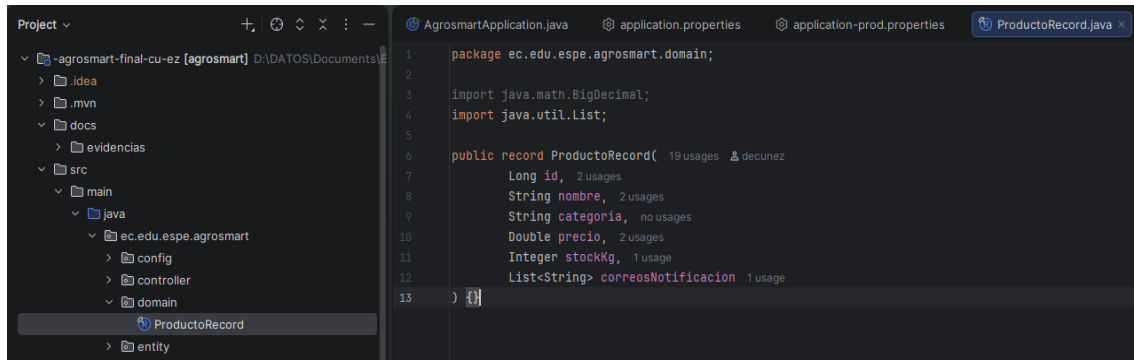
La tabla tbl_productos_base_86 se mapea correctamente con ProductoEntity e inserta los datos con DataSeeder.

```
mydatabase=# \d tbl_productos_base_86
Table "public.tbl_productos_base_86"
  Column          |          Type          | Collation | Nullable |          Default
-----+-----+-----+-----+-----
id_producto       | bigint                 |           | not null | generated by default as identity
categoria         | character varying(255) |           |         |
correos_notificacion | character varying(500) |           |         |
nombre_producto   | character varying(255) |           | not null |
precio_usd        | numeric(10,2)          |           |         |
stock_kg          | integer                |           | not null |
id                | bigint                 |           | not null | generated by default as identity
nombre            | character varying(255) |           |         |
precio            | double precision       |           |         |
Indexes:
    "tbl_productos_base_86_pkey" PRIMARY KEY, btree (id_producto)
    "uk9chn4gmukqkc2xpwstpnymknv" UNIQUE CONSTRAINT, btree (nombre_producto)
Referenced by:
    TABLE "producto_entity_correos_notificacion" CONSTRAINT "fkklfc7we8tur6h931f62j76p8" FOREIGN KEY (producto_entity_id) REFERENCES tbl_pr
oductos_base_86(id_producto)

mydatabase=# SELECT * FROM tbl_productos_base_86;
 29 | Flores |  | Orquídeas Mindo Premium |  | 120 | 29 |  | 22
 30 | Flores |  | Muestra Ramo Pruebas |  | 50 | 30 |  | 0
 31 | Flores |  | Flores Reserva Sin Notif |  | 80 | 31 |  | 15
(5 rows)

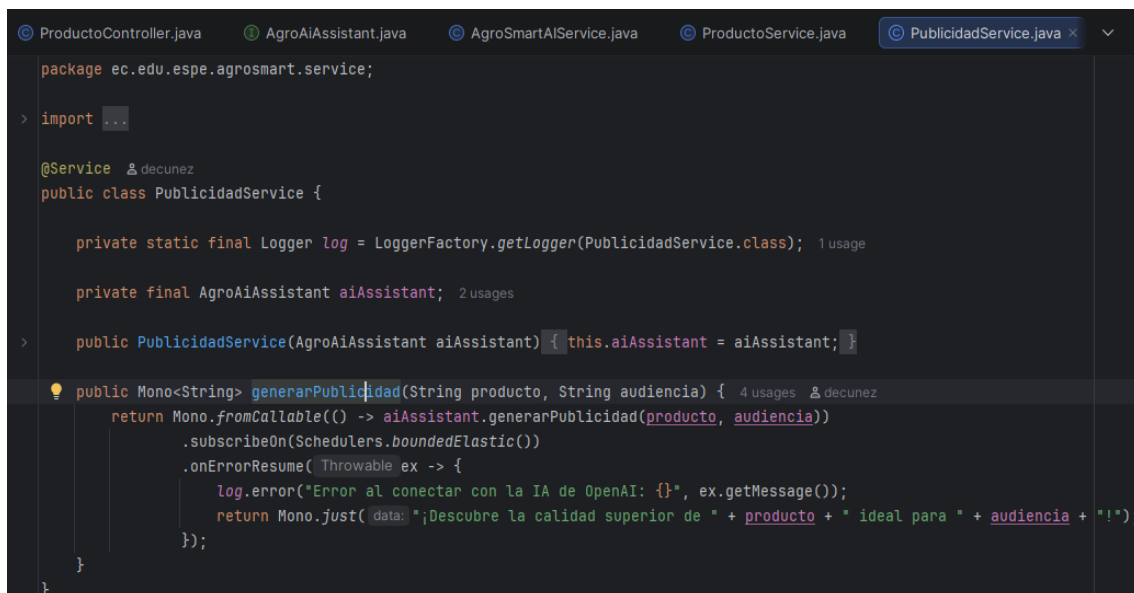
mydatabase=# \q
```

Uso de ProductoRecord (inmutable), separación limpia con ProductoEntity y mapeo funcional.



```
1 package ec.edu.espe.agrosmart.domain;
2
3 import java.math.BigDecimal;
4 import java.util.List;
5
6 public record ProductoRecord(
7     Long id,
8     String nombre,
9     String categoria,
10    Double precio,
11    Integer stockKg,
12    List<String> correosNotificacion
13 )
```

Flujo reactivo y aislamiento del bloqueo



```
package ec.edu.espe.agrosmart.service;

import ...

@Service
public class PublicidadService {

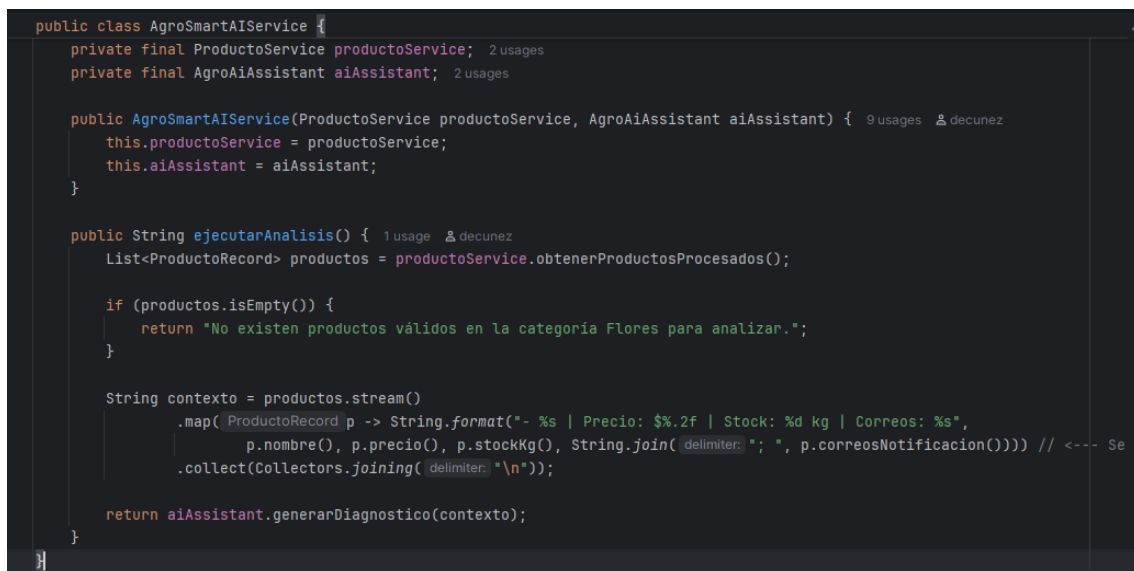
    private static final Logger log = LoggerFactory.getLogger(PublicidadService.class);

    private final AgroAiAssistant aiAssistant;

    public PublicidadService(AgroAiAssistant aiAssistant) { this.aiAssistant = aiAssistant; }

    public Mono<String> generarPublicidad(String producto, String audiencia) {
        return Mono.fromCallable(() -> aiAssistant.generarPublicidad(producto, audiencia))
            .subscribeOn(Schedulers.boundedElastic())
            .onErrorResume( Throwable ex -> {
                log.error("Error al conectar con la IA de OpenAI: {}", ex.getMessage());
                return Mono.just("¡Descubre la calidad superior de " + producto + " ideal para " + audiencia + "!");
            });
    }
}
```

Integración de IA LangChain4j



```
public class AgroSmartAIService {

    private final ProductoService productoService;
    private final AgroAiAssistant aiAssistant;

    public AgroSmartAIService(ProductoService productoService, AgroAiAssistant aiAssistant) {
        this.productoService = productoService;
        this.aiAssistant = aiAssistant;
    }

    public String ejecutarAnálisis() {
        List<ProductoRecord> productos = productoService.obtenerProductosProcesados();

        if (productos.isEmpty()) {
            return "No existen productos válidos en la categoría Flores para analizar.";
        }

        String contexto = productos.stream()
            .map(p -> String.format("- %s | Precio: $%.2f | Stock: %d kg | Correos: %s",
                p.nombre(), p.precio(), p.stockKg(), String.join(" ", p.correosNotificacion)))
            .collect(Collectors.joining("\n"));

        return aiAssistant.generarDiagnostico(contexto);
    }
}
```

```

package ec.edu.espe.agrosmart.service;

import ...

@AiService 5 usages 2 decunez
public interface AgroAiAssistant {

    String generarDiagnostico(String contexto); 1 usage 2 decunez

    @UserMessage(""" 3 usages 2 decunez
        |Redacta una frase publicitaria de máximo 100 caracteres para vender \
        |{{producto}} dirigido a {{audiencia}}.""")
    String generarPublicidad(@V("producto") String producto,
        @V("audiencia") String audiencia);
}

```

Endpoints mapeados devolviendo Flux y Mono, respondiendo texto plano y buscando por ID.

```

package ec.edu.espe.agrosmart.exception;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ResponseStatus;

@ResponseStatus(HttpStatus.NOT_FOUND) 5 usages 2 decunez
public class ProductoNoEncontradoException extends RuntimeException {
    public ProductoNoEncontradoException(Long id) { super("Producto no encontrado con id: " + id); }
}

package ec.edu.espe.agrosmart.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.util.Map;

@RestControllerAdvice 2 decunez
public class GlobalExceptionHandler {

    @ExceptionHandler(RuntimeException.class) no usages 2 decunez
    public ResponseEntity<Map<String, String>> handleRuntimeException(RuntimeException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(Map.of(
                k1: "codigo", ErrorCodigo.PRODUCTO_NO_ENCONTRADO.getCodigo(),
                k2: "mensaje", ex.getMessage()
            ));
    }
}

```

Pruebas Unitarias

```

13
14 @ExtendWith(MockitoExtension.class)
15 class PublicidadServiceTest {
16
17     @Mock
18     private AgroAiAssistant aiAssistant;
19
20     @InjectMocks
21     private PublicidadService publicidadService;
22
23     @Test
24     @DisplayName("generarPublicidad retorna la frase generada por la IA cuando no hay errores")
25     void generarPublicidad_Exit() {
26         when(aiAssistant.generarPublicidad( producto: "Rosas", audiencia: "Exportadores"))
27             .thenReturn( t "¡Rosas de alta calidad para exportación!");
28
29         StepVerifier.create(publicidadService.generarPublicidad( producto: "Rosas", audiencia: "Exportadores"))
30             .expectNext( t "¡Rosas de alta calidad para exportación!")
31             .verifyComplete();
32     }
33
34     @Test
35     @DisplayName("generarPublicidad activa el fallback onErrorResume si la IA falla")
36     void generarPublicidad_CuandoFallaIA_RetornaMensajeFallback() {
37         when(aiAssistant.generarPublicidad(anyString(), anyString()))
38             .thenThrow(new RuntimeException("Error de cuota o red de OpenAI"));
39
40         StepVerifier.create(publicidadService.generarPublicidad(anyString(), anyString()))
41             .expectNext( t "¡Rosas de alta calidad para exportación!")
42             .verifyComplete();
43     }
44 }
45
46 class ProductoServiceTest {
47     void obtenerProductosComercializables_FiltrarProductosInvalidos() {
48         StepVerifier.create(productoService.obtenerProductosComercializables())
49             .expectNextMatches( ProductoRecord p -> p.id().equals(1L) && p.nombre().equals("Rosas Rojas"))
50             .verifyComplete();
51     }
52
53     @Test
54     @DisplayName("buscarPorId retorna Mono con el ProductoRecord si el ID existe")
55     void buscarPorId_CuandoExiste_DevuelveProducto() {
56         when(productoRepository.findById(1L)).thenReturn(Optional.of(productoValido));
57
58         StepVerifier.create(productoService.buscarPorId(1L))
59             .expectNextMatches( ProductoRecord p -> p.id().equals(1L) && p.precio() == 12.50)
60             .verifyComplete();
61     }
62
63     @Test
64     @DisplayName("buscarPorId emite ProductoNoEncontradoException si el ID no existe")
65     void buscarPorId_CuandoNoExiste_LanzaExcepcion() {
66         when(productoRepository.findById(99L)).thenReturn(Optional.empty());
67
68         StepVerifier.create(productoService.buscarPorId(99L))
69             .expectError(ProductoNoEncontradoException.class)
70             .verify();
71     }
72 }

```

More tool windows | Application x | java in agrosmart x

default package 2 sec 999 ms 9 tests passed 9 tests total, 2 sec 999 ms

- ✓ AgroSmartControllerTo... 810 ms
 - ✓ GET /api/productos 2 sec 609 ms
 - ✓ GET /api/productos retor 151 ms
 - ✓ GET /api/agrosmart/public 50 ms
- ✓ ProductoServiceTest 149 ms
 - ✓ buscarPorId emite Produ 137 ms
 - ✓ obtenerProductosComerci 9 ms
 - ✓ buscarPorId retorna Mono 3 ms
- > AgrosmartApplicationTests 9 ms
- ✓ PublicidadServiceTest 31 ms
 - ✓ generarPublicidad retorna 17 ms
 - ✓ generarPublicidad activa e 14 ms

```

values
(?, ?)

Hibernate:
insert
into
    producto_entity_correos_notificacion
    (producto_entity_id, correos_notificacion)
values
    (?, ?)

Hibernate:
insert
into
    producto_entity_correos_notificacion
    (producto_entity_id, correos_notificacion)
values
    (?, ?)

2026-07-30T23:36:39.100-05:00 ERROR 5764 --- [agrosmart] [oundedElastic-1] e.e.e.a.service.PublicidadService : Error al conectar con la IA
2026-07-30T23:36:39.171-05:00 INFO 5764 --- [agrosmart] [ionShutdownHook] j.LocalContainerEntityManagerFactoryBean : Closing JPA EntityManagerFactory
2026-07-30T23:36:39.175-05:00 INFO 5764 --- [agrosmart] [ionShutdownHook] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Shutdown in
2026-07-30T23:36:39.183-05:00 INFO 5764 --- [agrosmart] [ionShutdownHook] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Shutdown com

Process finished with exit code 0

```